

CS 224n Assignment #2: word2vec (43 Points)

1 Written: Understanding word2vec (23 points)

Let's have a quick refresher on the word2vec algorithm. The key insight behind word2vec is that ‘a word is known by the company it keeps’. Concretely, suppose we have a ‘center’ word c and a contextual window surrounding c . We shall refer to words that lie in this contextual window as ‘outside words’. For example, in Figure 1 we see that the center word c is ‘banking’. Since the context window size is 2, the outside words are ‘turning’, ‘into’, ‘crises’, and ‘as’.

The goal of the skip-gram word2vec algorithm is to accurately learn the probability distribution $P(O|C)$. Given a specific word o and a specific word c , we want to calculate $P(O = o | C = c)$, which is the probability that word o is an ‘outside’ word for c , i.e., the probability that o falls within the contextual window of c .

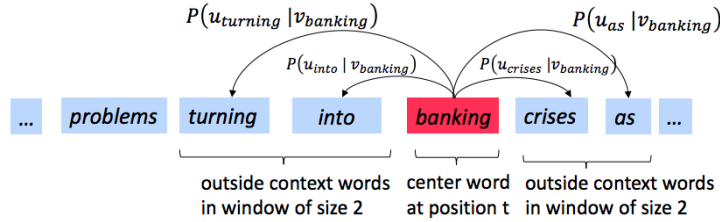


Figure 1: The word2vec skip-gram prediction model with window size 2

In word2vec, the conditional probability distribution is given by taking vector dot-products and applying the softmax function:

$$P(O = o | C = c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \quad (1)$$

Here, $\mathbf{u}_o$ is the ‘outside’ vector representing outside word o , and $\mathbf{v}_c$ is the ‘center’ vector representing center word c . To contain these parameters, we have two matrices, $\mathbf{U}$ and $\mathbf{V}$. The columns of $\mathbf{U}$ are all the ‘outside’ vectors $\mathbf{u}_w$. The columns of $\mathbf{V}$ are all of the ‘center’ vectors $\mathbf{v}_w$. Both $\mathbf{U}$ and $\mathbf{V}$ contain a vector for every $w \in \text{Vocabulary}$.¹

Recall from lectures that, for a single pair of words c and o , the loss is given by:

$$\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U}) = -\log P(O = o | C = c). \quad (2)$$

Another way to view this loss is as the cross-entropy² between the true distribution $\mathbf{y}$ and the predicted distribution $\hat{\mathbf{y}}$. Here, both $\mathbf{y}$ and $\hat{\mathbf{y}}$ are vectors with length equal to the number of words in the vocabulary. Furthermore, the k^{th} entry in these vectors indicates the conditional probability of the k^{th} word being an ‘outside word’ for the given c . The true empirical distribution $\mathbf{y}$ is a one-hot vector with a 1 for the true outside word o , and 0 everywhere else. The predicted distribution $\hat{\mathbf{y}}$ is the probability distribution $P(O|C = c)$ given by our model in equation (1).

- (a) (3 points) Show that the naive-softmax loss given in Equation (2) is the same as the cross-entropy loss between $\mathbf{y}$ and $\hat{\mathbf{y}}$; i.e., show that

¹Assume that every word in our vocabulary is matched to an integer number k . $\mathbf{u}_k$ is both the k^{th} column of $\mathbf{U}$ and the ‘outside’ word vector for the word indexed by k . $\mathbf{v}_k$ is both the k^{th} column of $\mathbf{V}$ and the ‘center’ word vector for the word indexed by k . **In order to simplify notation we shall interchangeably use k to refer to the word and the index-of-the-word.**

²The Cross Entropy Loss between the true (discrete) probability distribution p and another distribution q is $-\sum_i p_i \log(q_i)$.

$$-\sum_{w \in V_{ocab}} y_w \log(\hat{y}_w) = -\log(\hat{y}_o). \quad (3)$$

Your answer should be one line.

- (b) (5 points) Compute the partial derivative of $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ with respect to $\mathbf{v}_c$. Please write your answer in terms of $\mathbf{y}$, $\hat{\mathbf{y}}$, and $\mathbf{U}$.
- (c) (5 points) Compute the partial derivatives of $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ with respect to each of the ‘outside’ word vectors, $\mathbf{u}_w$ ’s. There will be two cases: when $w = o$, the true ‘outside’ word vector, and $w \neq o$, for all other words. Please write your answer in terms of $\mathbf{y}$, $\hat{\mathbf{y}}$, and $\mathbf{v}_c$.
- (d) (3 Points) The sigmoid function is given by Equation 4:

$$\sigma(\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{x}}} = \frac{e^{\mathbf{x}}}{e^{\mathbf{x}} + 1} \quad (4)$$

Please compute the derivative of $\sigma(\mathbf{x})$ with respect to $\mathbf{x}$, where $\mathbf{x}$ is a vector.

- (e) (4 points) Now we shall consider the Negative Sampling loss, which is an alternative to the Naive Softmax loss. Assume that K negative samples (words) are drawn from the vocabulary. For simplicity of notation we shall refer to them as $w_1, w_2, \dots, w_K$ and their outside vectors as $\mathbf{u}_1, \dots, \mathbf{u}_K$. Note that $o \notin \{w_1, \dots, w_K\}$. For a center word c and an outside word o , the negative sampling loss function is given by:

$$\mathbf{J}_{\text{neg-sample}}(\mathbf{v}_c, o, \mathbf{U}) = -\log(\sigma(\mathbf{u}_o^\top \mathbf{v}_c)) - \sum_{k=1}^K \log(\sigma(-\mathbf{u}_k^\top \mathbf{v}_c)) \quad (5)$$

for a sample $w_1, \dots, w_K$, where $\sigma(\cdot)$ is the sigmoid function.³

Please repeat parts (b) and (c), computing the partial derivatives of $\mathbf{J}_{\text{neg-sample}}$ with respect to $\mathbf{v}_c$, with respect to $\mathbf{u}_o$, and with respect to a negative sample $\mathbf{u}_k$. Please write your answers in terms of the vectors $\mathbf{u}_o$, $\mathbf{v}_c$, and $\mathbf{u}_k$, where $k \in [1, K]$. After you’ve done this, describe with one sentence why this loss function is much more efficient to compute than the naive-softmax loss. Note, you should be able to use your solution to part (d) to help compute the necessary gradients here.

- (f) (3 points) Suppose the center word is $c = w_t$ and the context window is $[w_{t-m}, \dots, w_{t-1}, w_t, w_{t+1}, \dots, w_{t+m}]$, where m is the context window size. Recall that for the skip-gram version of word2vec, the total loss for the context window is:

$$\mathbf{J}_{\text{skip-gram}}(\mathbf{v}_c, w_{t-m}, \dots, w_{t+m}, \mathbf{U}) = \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \mathbf{J}(\mathbf{v}_c, w_{t+j}, \mathbf{U}) \quad (6)$$

Here, $\mathbf{J}(\mathbf{v}_c, w_{t+j}, \mathbf{U})$ represents an arbitrary loss term for the center word $c = w_t$ and outside word w_{t+j} . $\mathbf{J}(\mathbf{v}_c, w_{t+j}, \mathbf{U})$ could be $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, w_{t+j}, \mathbf{U})$ or $\mathbf{J}_{\text{neg-sample}}(\mathbf{v}_c, w_{t+j}, \mathbf{U})$, depending on your implementation.

Write down three partial derivatives:

- (i) $\partial \mathbf{J}_{\text{skip-gram}}(\mathbf{v}_c, w_{t-m}, \dots, w_{t+m}, \mathbf{U}) / \partial \mathbf{U}$
(ii) $\partial \mathbf{J}_{\text{skip-gram}}(\mathbf{v}_c, w_{t-m}, \dots, w_{t+m}, \mathbf{U}) / \partial \mathbf{v}_c$

³Note: the loss function here is the negative of what Mikolov et al. had in their original paper, because we are doing a minimization instead of maximization in our assignment code. Ultimately, this is the same objective function.

(iii) $\partial \mathbf{J}_{\text{skip-gram}}(\mathbf{v}_c, w_{t-m}, \dots, w_{t+m}, \mathbf{U}) / \partial \mathbf{v}_w$ when $w \neq c$

Write your answers in terms of $\partial \mathbf{J}(\mathbf{v}_c, w_{t+j}, \mathbf{U}) / \partial \mathbf{U}$ and $\partial \mathbf{J}(\mathbf{v}_c, w_{t+j}, \mathbf{U}) / \partial \mathbf{v}_c$. This is very simple – each solution should be one line.

Once you're done: Given that you computed the derivatives of $\mathbf{J}(\mathbf{v}_c, w_{t+j}, \mathbf{U})$ with respect to all the model parameters $\mathbf{U}$ and $\mathbf{V}$ in parts (a) to (c), you have now computed the derivatives of the full loss function $\mathbf{J}_{\text{skip-gram}}$ with respect to all parameters. You're ready to implement **word2vec**!

2 Coding: Implementing word2vec (20 points)

In this part you will implement the word2vec model and train your own word vectors with stochastic gradient descent (SGD). Before you begin, first run the following commands within the assignment directory in order to create the appropriate conda virtual environment. This guarantees that you have all the necessary packages to complete the assignment.

```
conda env create -f env.yml
conda activate a2
```

Once you are done with the assignment you can deactivate this environment by running:

```
conda deactivate
```

- (a) (12 points) First, implement the sigmoid function in `word2vec.py` to apply the sigmoid function to an input vector. In the same file, fill in the implementation for the softmax and negative sampling loss and gradient functions. Then, fill in the implementation of the loss and gradient functions for the skip-gram model. When you are done, test your implementation by running `python word2vec.py`.
- (b) (4 points) Complete the implementation for your SGD optimizer in `sgd.py`. Test your implementation by running `python sgd.py`.
- (c) (4 points) Show time! Now we are going to load some real data and train word vectors with everything you just implemented! We are going to use the Stanford Sentiment Treebank (SST) dataset to train word vectors, and later apply them to a simple sentiment analysis task. You will need to fetch the datasets first. To do this, run `sh get_datasets.sh`. There is no additional code to write for this part; just run `python run.py`.

Note: The training process may take a long time depending on the efficiency of your implementation (an efficient implementation takes approximately an hour). Plan accordingly!

After 40,000 iterations, the script will finish and a visualization for your word vectors will appear. It will also be saved as `word_vectors.png` in your project directory. **Include the plot in your homework write up.** Briefly explain in at most three sentences what you see in the plot.

3 Submission Instructions

You shall submit this assignment on GradeScope as two submissions – one for “Assignment 2 [coding]” and another for “Assignment 2 [written]”:

- (a) Run the `collect_submission.sh` script to produce your `assignment2.zip` file.
- (b) Upload your `assignment2.zip` file to GradeScope to “Assignment 2 [coding]”.
- (c) Upload your written solutions to GradeScope to “Assignment 2 [written]”.